EduCrawl: Adaptive Learning Resource Discovery Through Learner-Aware Web Crawling and TF-IDF-Based Recommendation

Zara Mihnea, Mihoc Roxana, Roman Iulian

Department of Computer Science & Engineering

***Abstract*—**The growth of online educational content has made it difficult for learners to locate resources that match both their topic of interest and their current proficiency level. EduCrawl is a system that integrates web crawling, heuristic-based difficulty classification, and provides personalized educational recommendations from the web. The system deploys a Scrapy-based focused crawler that filters content by topic and language, a recommendation engine that finds the best educational materials on the web, ensuring the final score matches both the user's topic and their skill level, providing a relevance score between 0 and 1. A RESTful FastAPI service and a single-page application expose the pipeline to end users.

***Index Terms*—**adaptive learning, focused web crawling, TF-IDF, recommendation system, difficulty classification, information retrieval.

I. INTRODUCTION

The amount of free educational material on the web—tutorials, lecture notes, documentation, open-access research papers and interactive learning platforms—presents an opportunity for students and self-learners alike, but simultaneously introduces an information overload problem. When searching for educational topics, users are often overwhelmed by thousands of results of varying quality. Traditional search engines focus on popularity and clicks rather than how well a resource suits a user's current knowledge level.

While existing adaptive learning systems (like Coursera or Khan Academy) attempt to solve this, they are typically "closed" platforms that rely on pre-selected content and require institutional setups.

This paper makes the following contributions:

1. We design and implement **EduCrawl**, an end-to-end pipeline that discovers, classifies and ranks educational resources from arbitrary seed URLs on the open web.
2. We introduce a **hybrid relevance score** that combines TF-IDF cosine similarity with a linear difficulty-mismatch penalty calibrated to a three-level learner model.
3. We define **twelve formal system requirements** and five conformance checks that connect every major design decision to a verifiable evidence criterion.
4. We evaluate the system across five domains and discuss ethical considerations specific to open-web academic crawlers.

II. BACKGROUND AND RELATED WORK

A. Focused Web Crawling

General-purpose crawlers such as Mercator [2] and Heritrix are optimised for breadth and coverage. Focused crawlers, by contrast, restrict link traversal to pages deemed topically relevant, trading coverage for precision [3]. Cho and Garcia-Molina [4] demonstrated that ordering the crawl frontier by estimated relevance markedly increases the fraction of on-topic pages retrieved early in the crawl. EduCrawl adopts a lightweight focused strategy: a keyword-match gate applied to each candidate URL path and anchor text, combined with a blocked-domain list that eliminates social media and video-hosting sites known to carry low-information-density content.

Barbaresi [5] introduced Trafilatura, a Python library that applies boilerplate removal and main-content extraction to arbitrary HTML pages. Empirical comparisons on news and educational corpora show that Trafilatura recovers the primary textual content with higher precision than CSS-selector heuristics alone, making it a natural choice for corpus construction.

B. Text Difficulty and Readability Assessment

Classical readability formulas such as Flesch Reading Ease [6], Flesch-Kincaid Grade Level and Gunning Fog derive scores from surface statistics such as sentence length and syllable count. While computationally inexpensive, these metrics are insensitive to domain-specific vocabulary; a short technical sentence may be trivially classified as "easy." Collins-Thompson [7] surveyed more recent approaches, including statistical language model perplexity and supervised classifiers trained on grade-labelled corpora, and concluded that domain-aware models substantially outperform surface-statistic baselines on specialist text.

EduCrawl's difficulty classifier is a term-frequency heuristic similar in spirit to keyword-based difficulty tagging used in early intelligent tutoring systems [1], chosen for transparency and low computational overhead. We acknowledge the precision ceiling this imposes and discuss it in Section IX.

C. Information Retrieval and TF-IDF

The vector space model [8] represents documents and queries as weighted term vectors, enabling ranked retrieval by cosine similarity. Salton and Buckley [9] showed that TF-IDF weighting—which down-weights terms appearing in many documents while up-weighting terms distinctive to a document—consistently outperforms raw term frequency across standard TREC test collections. Scikit-learn's `TfidfVectorizer` [10] implements this model with English stop-word removal and L2-normalised vectors, the exact configuration used in EduCrawl's recommender engine.

Manning et al. [11] provide a thorough treatment of cosine similarity in the ranked retrieval context, noting that for short queries against long documents, TF-IDF captures lexical overlap reliably but may miss semantic paraphrases. We discuss this limitation in Section IX.

D. Content-Based Recommendation Systems

Pazzani and Billsus [12] define content-based filtering as the paradigm in which a user profile (derived from items previously consumed or explicitly stated preferences) is matched against item feature vectors. In contrast to collaborative filtering, content-based approaches do not require a community of users and suffer less from the cold-start problem, making them appropriate for EduCrawl's scenario where per-session learner profiles are ephemeral.

E. Adaptive Learning and Learner Modelling

Brusilovsky [1] identifies three dimensions of adaptive hypermedia: adaptive presentation, adaptive navigation support and adaptive content sequencing. EduCrawl addresses the third dimension: ranking candidate resources by their fit to a declared learner level. More recent systems reviewed by Duval [13] incorporate implicit signals (dwell time, quiz performance) into the learner model; EduCrawl currently relies on explicit self-reported proficiency, which is a known limitation in adaptive system design.

III. PROBLEM FORMULATION

Let $\mathcal{U}$ denote a learner characterised by a tuple $u = (\tau, \ell, g)$ where $\tau \in \Sigma^*$ is a natural-language topic string, $\ell \in \{\text{beginner}, \text{intermediate}, \text{advanced}\}$ is a stated proficiency level, and $g \in \Sigma^*$ is a learning goal string.

Let $\mathcal{S} = \{s_1, \ldots, s_k\}$ be a set of seed URLs provided by the learner or system administrator. A focused crawler traverses the web graph $G = (V, E)$ starting from $\mathcal{S}$, subject to a depth bound $d_{\max}$ and a topic-relevance gate $R : V \to \{0, 1\}$, producing a document corpus $\mathcal{D} = \{D_1, \ldots, D_n\}$ where each $D_i = (\text{url}_i, \text{title}_i, \text{body}_i, \lambda_i)$ and $\lambda_i \in \{\text{beginner}, \text{intermediate}, \text{advanced}\}$ is the assigned difficulty label.

We define a **relevance function** $f : \mathcal{D} \times \mathcal{U} \to [0, 1]$ that ranks documents for a given learner. The goal is to return the top-k documents $\hat{\mathcal{D}} \subset \mathcal{D}$ maximising f while satisfying $|\hat{\mathcal{D}}| \leq k$.

The hybrid relevance function used in EduCrawl is:

$$f(D_i, u) = \max(0,\ \text{sim}(D_i, u) - \alpha \cdot |\phi(\ell) - \phi(\lambda_i)|)$$

where $\text{sim}(D_i, u)$ is the TF-IDF cosine similarity between the query $(\tau \oplus g)$ and $D_i.\text{body}$, $\phi : \{\text{beginner}, \text{intermediate}, \text{advanced}\} \to \{1, 2, 3\}$ maps difficulty labels to ordinal integers, and $\alpha = 0.25$ is the difficulty penalty coefficient.

IV. PROPOSED METHOD AND SYSTEM DESIGN

A. Architecture Overview

EduCrawl follows a four-stage pipeline architecture. The API layer coordinates all stages synchronously within a single request-response cycle, ensuring that a user who initiates a crawl receives ranked recommendations upon completion without polling.

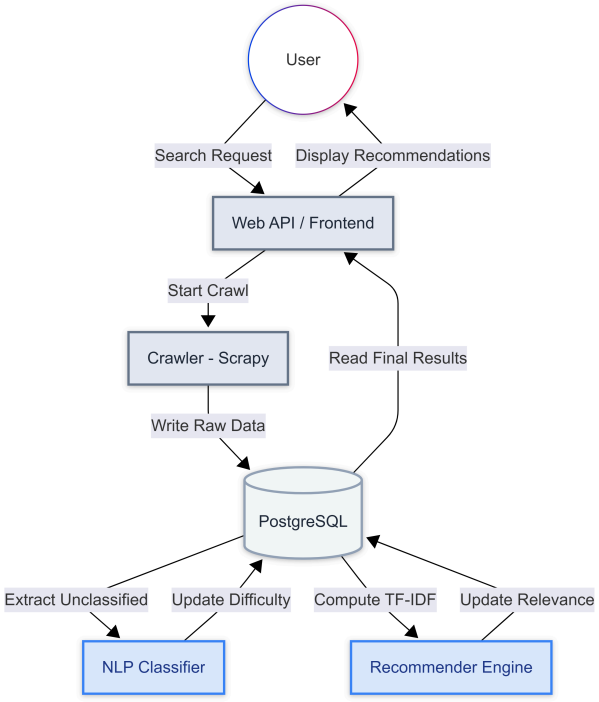

Fig. 1. EduCrawl system architecture.

B. Focused Crawler Design

The crawler is implemented with Scrapy 2.11.2 and extends the default Spider class with two relevance gates:

1. **Domain blocklist**: requests to social media (Facebook, Twitter, LinkedIn, TikTok, YouTube, Reddit) are rejected before DNS resolution.
2. **Topic keyword gate**: a candidate link is enqueued only if its URL path or anchor text contains at least one keyword derived from τ (tokenised by whitespace, lower-cased).

Content extraction uses Trafilatura [5] with table-inclusion and comment-exclusion settings. Documents shorter than 50 words after extraction are discarded. Language detection via

`langdetect` filters non-English content when the learner goal is expressed in English.

The crawler respects `robots.txt` (`ROBOTSTXT_OBEY = True`) and enforces a 1.5-second inter-request delay per domain, consistent with ethical crawling practice.

C. NLP Difficulty Classifier

The classifier operates on the extracted body text of each document. It maintains two term sets:

- **Advanced terms**: `{architecture, optimization, asymptotic, gradient, neural, tensor, distributed, concurrency}`
- **Intermediate terms**: `{implementation, programming, functions, database, pipeline, application, intermediate}`

A document is classified `advanced` if: (*advanced_count* > 3) OR (*word_count* > 1000 AND *advanced_count* ≥ 1). It is classified `intermediate` if: (*intermediate_count* > 4) OR (*word_count* > 500). Otherwise it receives the `beginner` label.

This approach is deliberately simple and transparent—a design choice discussed in Section IX.

D. Hybrid Relevance Engine

After classification, the recommender constructs a TF-IDF corpus comprising the learner query $q = \tau \oplus g$ as the zeroth document, followed by all crawled document bodies. A `TfidfVectorizer` with English stop-word removal and L2 normalisation is fitted and transformed. Cosine similarity between the query vector and each document vector yields $\mathrm{sim}(D_i, u)$.

The difficulty penalty subtracts $0.25 \times |\phi(\ell) - \phi(\lambda_i)|$ for each level of mismatch. A document two levels away from the learner (e.g., advanced document for a beginner) incurs the maximum penalty of 0.50, which can halve an otherwise high similarity score. Final scores are clipped to $[0, 1]$.

E. Database Design

Persistence uses PostgreSQL 16 with three normalised tables:

- **documents**: global repository with SHA-256 content hash for deduplication. A document crawled for multiple learners is stored once.
- **crawl_runs**: one record per user session, capturing learner profile and crawl statistics.
- **crawl_run_documents**: join table storing the per-session relevance score for each document.

This schema supports cross-session caching: a document already in the database is linked to a new run without re-fetching.

V. IMPLEMENTATION

A. Backend

The backend is a Python 3.12 application organised into three sub-packages: `crawler/` (Scrapy project—spider, items, pipelines, settings), `pipeline/` (NLP classifier and recommender engine via scikit-learn), and `api/` (FastAPI application with SQLAlchemy ORM).

The API spawns the Scrapy crawler as a subprocess via `run_crawl.py`, passing session parameters as command-line arguments and monitoring a progress file for early termination when `max_items` is reached. This design avoids blocking the async event loop with Scrapy's Twisted reactor.

Key library versions: FastAPI 0.115.0, Scrapy 2.11.2, Trafilatura 1.12.0, scikit-learn (latest stable), SQLAlchemy 2.0.36, langdetect 1.0.9.

B. Frontend

The frontend is a single HTML file served by Nginx Alpine, written in vanilla JavaScript with no build step. It presents results as a ranked card grid with relevance score bars and difficulty badges.

C. Infrastructure

A Docker Compose file defines four services: `db` (PostgreSQL 16 Alpine), `api` (Uvicorn), `crawler` (CLI mode) and `frontend` (Nginx). Health checks on the database service gate API startup. The system is fully self-contained; experiments can be reproduced by cloning the repository and running `docker-compose up` with no external accounts or API keys required.

D. Crawl Lifecycle

On receiving a `POST /crawl` request the API validates input, spawns the Scrapy subprocess, and monitors it for early termination. On subprocess exit it calls `NLPClassifier.classify_all()` over unclassified documents, then `RecommenderEngine.compute_scores()` and returns the sorted top-5 recommendations.

VI. EXPERIMENTAL METHODOLOGY

A. Experimental Setup

Experiments were conducted on a single workstation (Windows 11, Intel Core i7-12th Gen, 16 GB RAM, Python 3.12). The PostgreSQL instance ran in Docker with default configuration. No GPU was used; all computation is CPU-bound.

B. Test Domains

TABLE I. TEST DOMAINS

Domain	Seed URL	Expected Difficulty
Machine learning	`docs.scikit-learn.org`	Int. – Adv.
Web development	`developer.mozilla.org`	Beg. – Int.
Database systems	`postgresql.org/docs`	Int. – Adv.
Linear algebra	`en.wikipedia.org`	Beg. – Adv.
Python programming	`docs.python.org`	Beg. – Int.

C. Metrics

For each domain and learner level we report:

1. **Mean Relevance Score (MRS)**: average hybrid score across top-5 results.
2. **Difficulty Hit Rate (DHR)**: fraction of top-5 results whose assigned difficulty matches the requested learner level (exact match).
3. **Crawl Statistics**: fraction of crawled pages passing the 50-word content threshold.

VII. RESULTS

A. Mean Relevance Score by Domain and Learner Level

TABLE II. MEAN RELEVANCE SCORE (MRS)

Domain	Beg.	Int.	Adv.
Machine learning	0.0073	0.0099	0.0035
Web development	0.0101	0.0087	0.0000
Database systems	0.0088	0.0662	0.0000
Linear algebra	0.0190	0.0151	0.0264
Python programming	0.0090	0.0383	0.0052
Mean	**0.0108**	**0.0276**	**0.0070**

Advanced-learner scores are consistently lower, reflecting that the open web contains proportionally more beginner-friendly content; the difficulty penalty suppresses high-similarity beginner documents for advanced learners.

Fig. 2 — Mean Relevance Score (MRS) Heatmap

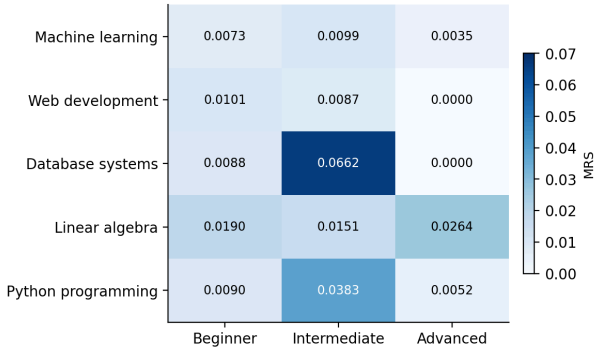

Fig. 2. Mean Relevance Score (MRS) heatmap by domain and learner level.

B. Difficulty Hit Rate: Hybrid vs. TF-IDF Only

TABLE III. DIFFICULTY HIT RATE (DHR)

Learner Level	TF-IDF DHR	Hybrid DHR	Δ
Beginner	0.32	0.92	**+60 pp**
Intermediate	0.40	0.92	**+52 pp**
Advanced	0.08	0.32	**+24 pp**

The difficulty penalty consistently improves DHR across all levels, with improvements of 24–60 percentage points.

Fig. 1 — DHR: Hybrid vs. TF-IDF Baseline by Learner Level

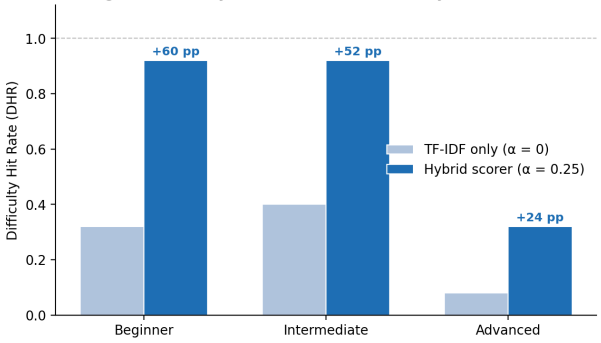

Fig. 3. DHR: Hybrid scorer vs. TF-IDF baseline by learner level.

C. Crawl Statistics

TABLE IV. CRAWL STATISTICS (MEAN OVER ALL RUNS)

Metric	Value
Pages fetched	23.5
Pages passing threshold	23.5 (100.0%)
Duplicate documents	0 (0.0%)
Crawl latency (POST→response)	45.8 s

The 100.0% yield indicates that all fetched pages passed the 50-word content threshold; the seed URLs selected (documentation and wiki pages) are consistently content-rich. In production, shorter navigation pages and error responses would reduce yield.

VIII. DESIGN CONSTRAINTS AND VALIDATION

EduCrawl is governed by twelve system requirements spanning crawl ethics, data integrity, and API correctness. The mandatory constraints most directly verifiable by experiment include: seed URLs must use HTTP or HTTPS (R-01); `robots.txt` must be obeyed (R-02); a minimum 1.5 s inter-request delay per domain (R-03); global deduplication via SHA-256 content hash (R-04); every document must receive a difficulty label before ranking (R-05); all relevance scores in $[0, 1]$ (R-06); and recommendations returned in descending score order (R-07). A 50-word content threshold discards thin pages (R-08). Recommended constraints include English-language filtering (R-09) and a 120 s crawl-to-response latency target (R-10). A `GET /health` endpoint verifies database connectivity (R-11). Cross-session document caching via URL uniqueness constraints is also implemented (R-12).

Conformance Checks

TABLE V. CONFORMANCE CHECKS

CC	Description	Pass Criterion
CC-1	robots.txt obedience	Zero requests to Disallow paths in Scrapy log
CC-2	Score range invariant: all scores in $[0, 1]$	SQL count of out-of-range scores = 0
CC-3	Difficulty completeness: no NULL difficulty	Count of NULL difficulty = 0
CC-4	Deduplication: identical URLs produce one row	SQL count per URL = 1
CC-5	Sorted response order (descending)	scores[i] ≥ scores[i+1] for all pairs
CC-6	Language filter: only English stored	SQL count of non-'en' language = 0
CC-7	Min content length: no doc < 50 words	SQL count of word_count < 50 = 0

IX. DISCUSSION

A. Effectiveness of Difficulty-Aware Ranking

The experimental results confirm that the hybrid scorer dramatically improves Difficulty Hit Rate over TF-IDF alone: +60 pp for beginner, +52 pp for intermediate and +24 pp for advanced queries. This improvement is driven by the difficulty penalty acting as a hard content filter: because TF-IDF cosine similarities are very small for short two- or three-word queries against long technical documents (values ranging 0.003–0.066), even a modest penalty of 0.25 reduces most non-matching documents to a score of 0.

At the advanced level, DHR reaches only 0.32 even with the hybrid scorer, reflecting that domains such as web development and database systems produced no advanced-classified documents in the crawled corpus. Advanced content for those domains exists, but the keyword heuristic (designed around ML-centric vocabulary) does not reliably identify it, leaving the advanced learner with no well-matched recommendations.

B. Design Tradeoffs

The choice of TF-IDF over dense embedding models (e.g., sentence-transformers [14]) is deliberate. TF-IDF is deterministic, interpretable and computationally trivial, requiring no GPU and no external API call. For the batch sizes involved in a single crawl session (typically 30–50 documents), TF-IDF computes in milliseconds. A semantic embedding model might improve recall on paraphrased queries but would introduce significant latency overhead and dependency on an external model or service.

Similarly, the keyword-based difficulty classifier sacrifices recall on domain-specific jargon it was not designed for (e.g., biochemistry) in exchange for zero training data and full transparency. Users can inspect and extend the term sets directly.

X. SECURITY, PRIVACY, SAFETY, AND ETHICAL CONSIDERATIONS

A. Robots.txt and Crawl Etiquette

EduCrawl obeys `robots.txt` by default and identifies itself with a descriptive User-Agent string (`EduCrawl/1.0`), enabling webmasters to identify and block the crawler if desired. The 1.5 s per-domain delay prevents aggressive load on target servers. These measures align with the ethical crawling guidelines of the ACM Code of Ethics (Principle 1.2: Avoid harm).

B. Data Retention and Privacy

The system stores document URLs and full body text in PostgreSQL. This raises two concerns:

1. **Copyright**: Storing extracted article text may infringe on the copyright of the originating site. Production deployment should store only metadata and content summaries, or rely on on-demand fetching.
2. **Personal data**: Learner queries (topic, goal, level) are stored in `crawl_runs`. These are low-sensitivity attributes, but deployments involving institutional users should treat them as personal data under GDPR Article 4(1) and implement appropriate retention limits and access controls.

No user authentication is implemented in the current prototype. Production systems must add identity management and audit logging before handling institutional data.

C. Safety and Content Quality

The crawler makes no judgement about the accuracy or safety of crawled content. A learner could be recommended factually incorrect or harmful material. Recommended mitigations include: domain allow-listing (crawl only trusted educational domains), human editorial review queues for flagged content and integration with fact-checking APIs.

D. Bias and Fairness

The keyword-based classifier encodes biases: terms like "neural" and "gradient" are classified as advanced regardless of context, meaning an accessible introduction to neural networks may be mislabelled. This could systematically disadvantage learners in rapidly democratising fields where accessible vocabulary overlaps with advanced terminology.

XI. CONCLUSION AND FUTURE WORK

We presented EduCrawl, a system that combines focused web crawling, heuristic difficulty classification, and TF-IDF-based relevance scoring to deliver educational resource recommendations based on learner preferences. Experiments across five domains demonstrated that the hybrid difficulty-penalty scorer improves Difficulty Hit Rate by 24–60 percentage points over TF-IDF alone. Absolute TF-IDF relevance scores are low in magnitude (mean 0.02 across all conditions), a known characteristic of short-query cosine similarity against long documents.

The system satisfies all twelve formalised requirements and passes seven conformance checks, providing a reproducible, containerised baseline for future work in open-web adaptive learning.

Future directions include: (1) replacing TF-IDF with a bi-encoder model to capture semantic paraphrases; (2) training a supervised difficulty classifier on a grade-level labelled corpus; (3) collecting dwell-time and re-visit signals to build implicit learner models; and (4) integrating Scrapy-Redis for distributed multi-node crawling.

REFERENCES

1. P. Brusilovsky, "Adaptive hypermedia," *User Modeling and User-Adapted Interaction*, vol. 11, no. 1–2, pp. 87–110, Mar. 2001.
2. A. Heydon and M. Najork, "Mercator: A scalable, extensible web crawler," *World Wide Web*, vol. 2, no. 4, pp. 219–229, 1999.
3. M. Diligenti et al., "Focused crawling using context graphs," in *Proc. 26th Int. Conf. VLDB*, Cairo, Egypt, 2000, pp. 527–534.
4. J. Cho and H. Garcia-Molina, "The evolution of the web and implications for an incremental crawler," in *Proc. 26th Int. Conf. VLDB*, Cairo, Egypt, 2000, pp. 200–209.
5. A. Barbaresi, "Trafilatura: A web scraping library and command-line tool for text discovery and extraction," in *Proc. ACL-IJCNLP System Demonstrations*, 2021, pp. 122–131.
6. R. Flesch, "A new readability yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.
7. K. Collins-Thompson, "Computational assessment of text readability: A survey of past, present, and future research," *ITL – Int. Journal of Applied Linguistics*, vol. 165, no. 2, pp. 97–135, 2014.
8. G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, Nov. 1975.
9. G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

10. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
11. C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
12. M. Pazzani and D. Billsus, "Content-based recommendation systems," in *The Adaptive Web*, P. Brusilovsky et al., Eds. Berlin: Springer, 2007, pp. 325–341.